Lore: Integrated product and engineering strategy

I've reviewed the proposed architecture against the Lore product we have developed so far.

My recommendation is to adopt the standards-first approach, but not make Lore dependent on Backstage, a graph database, or a voice-only workflow.

The proposal has several excellent ideas, particularly:

- Using open standards instead of creating proprietary integration formats.
- Connecting knowledge to actual engineering events.
- Capturing the reasoning behind code changes through short voice debriefs.
- Preserving source provenance and permissions.
- Moving from simple RAG to relationship-aware retrieval.
- Positioning Lore around engineering continuity rather than generic AI chat.

However, it also introduces considerable complexity too early. Building Kafka, Temporal, Neo4j, LiveKit, WorkOS, OPA, a VS Code extension and a multi-cloud enterprise architecture simultaneously would be a risky first implementation.

We should preserve the long-term architecture while choosing a much smaller initial system that can become a commercially credible product.

1. What I would keep, modify and reject

1. Architecture proposal assessment

Proposal	My assessment	Decision for Lore
Backstage `catalog-info.yaml`	Excellent for software catalog metadata	Adopt as an optional native integration
OpenTelemetry	Excellent for service traces, metrics and logs	Adopt for operational context

OpenLineage	Useful for data lineage and pipeline relationships	Support for data-platform customers
GitHub / GitLab events	Valuable context for engineering knowledge	Make this the first integration
Kafka / Redpanda	Useful for high-volume event streaming	Defer until event volume justifies it
Temporal	Useful for durable, long-running workflows	Introduce when automated capture workflows become complex
PostgreSQL + pgvector	Strong foundation for relational and semantic knowledge	Adopt
Neo4j / Memgraph	Useful for advanced relationship queries	Defer; start with PostgreSQL relationships
LiveKit	Strong option for real-time voice experiences	Evaluate as the voice infrastructure
Deepgram + Cartesia / ElevenLabs	Viable speech components	Keep provider-independent
WorkOS	Useful for enterprise identity and directory sync	Adopt when enterprise customers require it
OPA	Useful for centralized authorization policies	Defer; implement clear permission enforcement first

VS Code extension	Valuable contextual experience	Build after the web mentor is reliable
BYOC / self-hosting	Important for certain regulated customers	Enterprise roadmap, not initial release

A few important technical corrections:

Backstage is not a universal company knowledge standard. Its catalog descriptor is a structured format for software entities such as Components, APIs, Resources, Systems, Users and Groups. That makes it a useful integration contract, not a requirement that every Lore customer must adopt Backstage.

GitHub+1

OpenTelemetry and OpenLineage solve different problems. OpenTelemetry provides vendor-neutral observability signals—traces, metrics and logs. It should help Lore understand system behavior, not serve as the entire knowledge model.

OpenTelemetry+1
OpenLineage should be an optional adapter for data-job and dataset lineage.

LiveKit is a credible voice architecture choice. Its Agents framework supports real-time audio, text, model-provider flexibility and agent orchestration. It can support Lore's voice interviewer and mentor without requiring us to assemble every WebRTC component ourselves.

LiveKit Documentation+2

Temporal is useful, but not mandatory on day one. Its durable execution model is designed for workflows that must resume after failures. It becomes valuable when Lore schedules interviews, waits for approvals, retries ingestion and coordinates multiple systems.

Temporal Documentation+1

One other distinction: Spotify Backstage is a software catalog and developer portal ecosystem. Airbnb's Minerva is associated with data infrastructure. They are useful architectural inspirations, but we should not assume they share one universal manifest or that either alone solves institutional memory.

2. The architecture I recommend for Lore

The central architectural decision should be:

Build a canonical Lore Knowledge Model, then connect external systems to it through adapters.

Do not make Backstage, GitHub, Slack or any other external platform the source of truth for Lore's entire data model.

External systems describe their own data. Lore maps that data into a common internal representation.

2.1 Five architectural layers

Layer 1 — Source Connectors

Where organizational knowledge originates

GitHub · GitLab · Backstage · Slack · Teams · Jira · Confluence · Drive · OpenTelemetry · OpenLineage · Voice · Documents

Layer 2 — Ingestion & Normalization

Turn external information into a common format

Webhooks · Polling · Parsers · Event normalization · Deduplication · Permission mapping

Layer 3 — Lore Knowledge Core

The canonical organizational memory

PostgreSQL · pgvector · Object storage · Knowledge records · Relationships · Provenance · Access policies

Layer 4 — Intelligence Engine

Understand, retrieve, explain and preserve knowledge

RAG · Graph-assisted retrieval · Voice AI · Knowledge extraction · Decision intelligence · Gap detection

Layer 5 — Experience & Governance

How employees and administrators use Lore

Web workspace · Ask Lore · Capture Studio · IDE extension · Admin console · Audit and analytics

Security, tenancy, observability and authorization should be cross-cutting capabilities across all five layers, not a separate feature added at the end.

2.2 The most important decision: Lore's canonical knowledge schema

I would design the internal model around these entities:

Entity	What it represents
Organization	A customer tenant
User	An employee or contributor
Team	A group within an organization
System	A software service, application or operational system
Project	A project or initiative
Document	A source document or knowledge artifact
Conversation	An interview or recorded discussion
Decision	A decision and its reasoning
Procedure	A repeatable workflow
Incident	A failure, event or lesson learned

Event	A source-system activity such as a PR merge
Knowledge item	A reviewed, reusable piece of organizational knowledge
Evidence	The excerpt, recording or external source supporting a claim
Relationship	A connection between any relevant entities

A knowledge record should not be just a title and a vector embedding.

It should have fields such as:

```
None
{
  "id": "knowledge_001",
  "tenant_id": "org_001",
  "type": "architectural_decision",
  "title": "Custom retry mechanism in payment service",
  "summary": "Why the service uses a custom retry strategy.",
  "context": "Payment gateway timeout handling",
  "reasoning": "Captured explanation of the design choice.",
  "applicability": {
    "system": "payment-service",
    "version": "current",
    "environment": "production"
  },
  "contributors": ["user_001"],
  "source_refs": [
    "github_pr_412",
    "voice_session_023"
  ],
  "status": "approved",
  "visibility": "team",
  "created_at": "2026-10-31T08:00:00Z",
  "review_due_at": null
}
```

This is a proposed internal representation, not an official Backstage schema.

How external standards fit

Standard	Lore's adapter maps it to
Backstage Component	System / software service
Backstage API	API entity
Backstage Resource	Infrastructure resource
Backstage User / Group	User / Team
GitHub PR	Event / Change / Evidence
GitHub commit	Event / Change
OpenTelemetry trace	Operational evidence
OpenLineage job / dataset	Data pipeline / Dataset / Relationship
Voice interview	Conversation / Evidence / Knowledge item
Document	Document / Evidence / Knowledge item

This approach means Lore can support organizations with Backstage, without Backstage, or with their own internal catalog.

2.3 Start with a modular monolith

I would not start with microservices.

Use a modular backend with clear internal boundaries:

```
None
lore/
├── frontend/
│   └── web/
├── backend/
│   └── app/
│       ├── auth/
│       ├── tenants/
│       ├── users/
│       ├── connectors/
│       ├── ingestion/
│       ├── normalization/
│       ├── knowledge/
│       ├── conversations/
│       ├── extraction/
│       ├── retrieval/
│       ├── voice/
│       ├── governance/
│       ├── analytics/
│       └── audit/
├── workers/
│   ├── ingestion_worker/
│   ├── embedding_worker/
│   └── workflow_worker/
├── database/
│   ├── migrations/
│   └── seed/
├── integrations/
│   ├── github/
│   ├── backstage/
│   ├── slack/
│   ├── jira/
│   ├── opentelemetry/
│   └── openlineage/
└── docs/
    ├── architecture/
    ├── api/
```

```
    └── security/
```

This gives us separation of concerns without the deployment and debugging overhead of many independent services.

3. How Lore should actually work

I would build three interconnected intelligence loops.

Loop A — Passive knowledge ingestion

New capability

```
None
GitHub / GitLab / Documentation
              |
              v
        Connector event
              |
              v
       Normalize event
              |
              v
     Extract relevant context
              |
              v
      Link to existing entities
              |
              v
      Store evidence in Lore
              |
              v
    Identify possible knowledge gap
              |
              v
     Suggest expert debrief
```

Example:

A developer merges PR #412.

Lore extracts:

- Author.
- Repository.
- Changed files.
- Commit message.
- PR description.
- Review comments.
- Related service.
- Linked incident or issue, if available.

Lore does not automatically treat the PR as complete knowledge.

Instead, it identifies whether the change needs an explanation.

Loop B — Active knowledge capture

Existing core capability, expanded

This is where the original Lore concept remains central.

1. Lore identifies an event that may require explanation.
2. It suggests a short debrief to the contributor.
3. The contributor accepts or declines.
4. Lore starts a voice interview.
5. It asks questions using the actual PR, incident or project context.
6. It extracts decisions, alternatives, risks and gotchas.
7. The contributor reviews the extracted information.
8. Approved knowledge is linked to the relevant system and event.

Example prompt:

> "Your change to the authentication middleware was merged. Could you explain why the previous approach was insufficient and what someone maintaining this code should know?"

Important: Do not automatically initiate voice calls based on every event. Start with opt-in invitations, configurable triggers and sensible frequency limits. An engineer should never feel that Lore is monitoring or interrogating them.

Loop C — Contextual knowledge consumption

Core product

```
None
New employee asks a question
               |
               v
```

```
      Understand intent
             |
             v
Identify system / project / file
             |
             v
     Retrieve permissions
       filtered knowledge
             |
             v
  Search relevant evidence
             |
             v
   Combine graph relationships
             |
             v
    Generate grounded answer
             |
             v
      Display citations
             |
             v
 Offer expert or source escalation
```

This is the difference between a conventional vector-search chatbot and a useful organizational mentor.

A question about a specific code path should ideally retrieve:

- The service.
- The relevant commit.
- The pull request.
- The original decision.
- The expert's explanation.
- Related incidents.
- The applicable system version.

Do not assume the graph always improves results. Evaluate graph-assisted retrieval against hybrid keyword and vector retrieval on a fixed test set.

4. Recommended technology stack

This is the stack I would choose for a product designed to grow into enterprise deployments.

Component	Recommended choice	Reason
Frontend	Next.js + TypeScript	Mature SaaS application development
UI	Tailwind + shadcn/ui	Consistent component system
Backend	Python + FastAPI	AI, ingestion and API development
Database	PostgreSQL	Canonical data and relationships
Vector search	pgvector	Semantic retrieval
Object storage	S3-compatible storage	Audio, documents and source files
Voice infrastructure	LiveKit Agents	Realtime voice sessions
Speech services	Pluggable STT/TTS providers	Provider flexibility
LLM	Provider abstraction	Avoid vendor lock-in
Embeddings	Pluggable embedding model	Flexible retrieval

Background jobs	Redis + Celery initially	Ingestion and extraction tasks
Workflow orchestration	Temporal later	Durable multi-step processes
Event streaming	Webhooks + database outbox initially	Reliable event ingestion
Event streaming at scale	Redpanda / Kafka	High-volume event processing
Authentication	Auth provider initially; WorkOS for enterprise SSO	Identity and organization management
Authorization	PostgreSQL RLS + application policies	Tenant and resource access
Observability	OpenTelemetry	Traces, metrics and logs
Deployment	Docker + managed cloud infrastructure	Reproducible deployments
Graph database	Neo4j later, if justified	Advanced graph traversal

Why I would delay Neo4j

PostgreSQL can initially store relationships using relational tables and recursive queries.

For example:

```
None
Engineer
   |
```

```
   | AUTHORED
   v
Pull Request
   |
   | MODIFIES
   v
Software Service
   |
   | RELATED_TO
   v
Architectural Decision
   |
   | EXPLAINED_BY
   v
Voice Interview
```

This is sufficient to validate Lore's relationship model.

Only introduce a dedicated graph database when real customer workloads demonstrate that graph traversal, query performance or graph analytics justify the additional infrastructure.

Why I would use LiveKit rather than immediately combining several voice vendors

LiveKit already provides an agent framework, real-time media transport, model integrations and deployment options. We can retain flexibility over the underlying speech and language models.

LiveKit Documentation+2

I would build a `VoiceProvider` abstraction so we can change providers without rewriting Capture Studio or the mentor interface.

5. The most important enterprise requirement: permissions

I would make this a foundational part of the architecture, not a Phase 4 addition.

Imagine:

- An employee can access the public engineering repository.

- A contractor can access only one project.
- A security engineer can access the security repository.
- An HR employee can access employee-related information.

Lore must respect these boundaries.

Recommended authorization flow

User submits a question

Identify user and tenant

Resolve resource permissions

Filter retrieval candidates

Retrieve authorized evidence only

Generate answer and citations

Important implementation requirements:

1. Every tenant-owned record has a tenant identifier.
2. Every source has an access-control mapping.
3. Permission filtering happens before context reaches the LLM.
4. Revoked source access is reflected in Lore's retrieval system.
5. Answers and citations must not expose unauthorized metadata.
6. Audit records must capture the resources used to produce an answer.
7. Test cross-tenant access and permission revocation explicitly.

PostgreSQL Row-Level Security is useful, but it is not a complete security strategy by itself. The application, workers, storage layer, vector retrieval and external integrations all need consistent authorization.

Also, avoid promising that all data is never used for training until the actual model-provider contracts and configuration support that claim.

6. The complete development roadmap

I would plan this as a 6-month product development programme, followed by a customer validation and enterprise-readiness programme.

The schedule assumes a small, capable engineering team. It is not a guaranteed delivery timeline.

Month 1

Foundation and canonical knowledge model

- Multi-tenant database design.
- User, team and organization management.
- Knowledge entity schema.
- Source provenance.
- Authentication and basic authorization.
- Public website and authenticated workspace.
- Initial Ask Lore and Knowledge Library.

Month 2

Voice and knowledge intelligence

- LiveKit voice integration.
- Adaptive interviewer.
- Transcript storage.
- Structured knowledge extraction.
- Expert review and approval.
- pgvector retrieval.
- Source-grounded mentor.
- Unknown-answer handling.

Month 3

Engineering context integration

- GitHub App.
- Repository and PR ingestion.
- Commit and author metadata.
- Backstage catalog adapter.
- Canonical system and service mapping.
- Relationship storage.
- Event-triggered debrief suggestions.

Month 4

Organizational intelligence

- Expert directory.
- Decision history.
- Knowledge collections.
- Knowledge graph interface.
- Knowledge gaps.
- Review and freshness workflows.
- Onboarding journeys.

Month 5

Enterprise integrations and governance

- Slack / Teams.

- Jira and Confluence.
- Permission synchronization.
- Audit logs.
- Enterprise SSO.
- Data retention and deletion.
- Usage analytics.
- OpenTelemetry instrumentation.

Month 6

Reliability and commercial readiness

- Security testing.
- Backup and recovery.
- Tenant-isolation tests.
- Performance and load testing.
- Billing and subscription management.
- Customer onboarding.
- Documentation and API.
- Pilot deployment with real engineering teams.

What should be tested at every stage?

Area	Success measure
Knowledge extraction	Expert acceptance and correction rates
Retrieval	Relevant source in top results
Grounding	Answer claims supported by retrieved evidence
Voice	Successful session completion and latency
Integrations	Sync success and source freshness
Security	No unauthorized retrieval

Knowledge value	Reduced time to find reliable answers
Adoption	Repeat weekly active usage
Onboarding	Time to complete selected onboarding tasks

Set actual target thresholds only after establishing baseline measurements with pilot users.

7. First integration strategy: don't build 50 connectors

This is where I strongly agree with the standards-first proposal.

However, I would implement connectors in this order:

Github Black Icons – Free Download SVG, PNG, GIF

1. GitHub — first priority

Repository metadata, pull requests, commits, issues, authorship and code-change context.

First connector
Build your own Technical Documentation Portal using BackStage — Part 4 | by Susovan Panja | Medium

2. Backstage — catalog integration

Import services, APIs, resources, owners and system relationships. Make the integration optional.

New information about Slack's 2015 security incident | Slack

3. Slack / Microsoft Teams

Capture authorized discussions, deliver debrief invitations and surface Lore answers.

Jira Logo .PNG and .AI, .EPS, .CDR, .PDF, .SVG - iconLogoVector

4. Jira / Confluence

Link incidents, tickets, project discussions and documentation.

Opentelemetry Logo Png | Opentelemetry Logo Vector - freepng

5. OpenTelemetry

Add traces, metrics and logs as operational context for incidents and service knowledge.

openlineage

6. OpenLineage

Add data jobs, datasets and lineage for data engineering customers.

Connector engineering principle

Every connector should implement a common internal interface:

```
None
class KnowledgeConnector:    def authenticate(self):        pass
def validate_permissions(self):        pass    def
discover_sources(self):        pass    def fetch_changes(self,
cursor=None):        pass    def normalize(self, raw_event):
pass    def sync_permissions(self):        pass    def
health_check(self):        pass
```

The connector should translate its source-specific information into Lore's canonical model.

This makes future integrations much less expensive.

8. The product feature that could become Lore's strongest differentiator

I would introduce something called Knowledge Continuity Engine.

It connects three capabilities:

1. Understand what is changing in an organization.
2. Identify where important context is missing.
3. Proactively help capture that context from the right person.

For example:

Knowledge Continuity Engine

Concept

New engineering change

A significant change was merged into the payment service.

Context assessment

Lore finds the change, related decisions and existing documentation.

Knowledge gap detected

The reason for the retry mechanism is not explained in the available sources.

Suggested expert debrief

Lore invites the author to explain the decision in a short conversation.

Verified organizational memory

The explanation is reviewed, linked to the PR and made available to authorized employees.

This is much stronger than simply asking engineers to upload documents.

It makes Lore an active participant in maintaining organizational memory.

9. GTM: What I would change in the proposal

The proposed engineering-continuity positioning is good. I would retain it as the first market entry, but make the target customer more specific.

Initial ideal customer profile

Start with:

- B2B software companies.
- Approximately 30–300 engineers.
- Multiple repositories and services.
- Distributed or growing engineering teams.
- Frequent onboarding.
- Legacy systems and undocumented decisions.
- Engineering managers who recognize knowledge concentration risks.

Why not target every company immediately?

Because manufacturing, healthcare, financial services and software engineering have very different systems, security requirements and knowledge structures.

Prove the engineering use case first, then extend the same underlying model into other industries.

Recommended positioning

Lore — The Organizational Memory and Engineering Continuity Platform

Short positioning:

> Lore captures the reasoning behind engineering decisions, connects it to the systems and code it affects, and makes that knowledge accessible through a source-grounded AI mentor.

This is more defensible than "AI chatbot for documentation."

The free entry point

I like the proposed repository scanner, but I would not make a potentially misleading bus-factor score the centerpiece.

Instead, offer a Repository Knowledge Map:

- What services exist?
- Which teams own them?
- Which repositories are connected?
- Where are the important dependencies?
- Which systems have limited documentation?
- Which decisions are linked to code changes?
- Where could additional expert context be valuable?

The free tool should provide value even if the user never purchases the voice product.

Then introduce the paid capabilities:

- Automated knowledge debriefs.
- Voice mentor.
- Knowledge governance.
- Team-level intelligence.
- Enterprise permissions and integrations.

10. Pricing: revise the proposed model

I would not publish the proposed $29 / $59 per-engineer pricing as a final decision yet.

There are several issues:

- Voice processing and LLM usage create variable costs.

- Charging every engineer may discourage adoption.
- Some users contribute knowledge, while many primarily consume it.
- Enterprise identity, storage and deployment costs are different from ordinary seat costs.

I would test a hybrid model.

Plan	Proposed structure	Target
Free / Developer	Repository discovery with limited knowledge indexing	Individual developers and small teams
Team	Base workspace fee + active contributor seats	Small engineering teams
Business	Workspace fee + seats + usage allowance	Growing engineering organizations
Enterprise	Annual contract, custom security and deployment options	Large organizations

Potential paid usage dimensions:

- Active contributors.
- Knowledge storage.
- Voice interview minutes.
- AI query allowance.
- Integration count or source volume.
- Retention and advanced governance.

These are pricing hypotheses. Validate willingness to pay and actual infrastructure cost before setting numbers.

11. What the first 30 days should actually look like

The suggested 30-day checklist is too ambitious if it includes a production-ready repository scanner, graph engine, real-time voice platform and enterprise architecture.

I would narrow it to one convincing end-to-end engineering use case.

Week	Deliverable
Week 1	Canonical schema, GitHub App, repository metadata and PR ingestion
Week 2	Knowledge extraction, PostgreSQL storage, source linking and basic retrieval
Week 3	Voice debrief based on a real PR, transcript and expert approval
Week 4	Ask Lore with grounded answers, source references and a polished product demonstration

The demonstration scenario

Use a real or synthetic repository.

1. A developer merges a change.
2. Lore detects the PR.
3. Lore identifies missing explanatory context.
4. The developer accepts a debrief invitation.
5. Lore interviews them for a few minutes.
6. It creates an architectural decision record.
7. The developer approves it.
8. A new engineer asks why that code was implemented that way.
9. Lore answers and links directly to the PR and explanation.
10. An unrelated question receives an honest insufficient-evidence response.

This demonstrates the entire product philosophy in one coherent workflow.

12. The final integrated product roadmap

Here is how I would structure the full Lore product.

Product area	Initial release	Long-term capability
Ask Lore	Grounded text and voice mentor	IDE-aware GraphRAG mentor
Capture Studio	Voice and document capture	Event-triggered expert debriefs
Knowledge Hub	Structured knowledge library	Connected organizational memory
Integrations	GitHub	Multi-system connector ecosystem
Standards	Canonical internal model	Backstage, OTel, OpenLineage adapters
Knowledge Graph	Relational relationships	Advanced graph intelligence
Governance	Approval and provenance	Enterprise policy and audit
Intelligence	Search and source retrieval	Knowledge continuity engine
Onboarding	Curated knowledge collections	Personalized role-based learning
Deployment	Multi-tenant SaaS	Enterprise isolation / BYOC options

Useful technical references

These are the official resources I would use when designing the integration layer:

- Backstage catalog descriptor format
- OpenTelemetry documentation

- OpenLineage documentation
- LiveKit Agents
- Temporal documentation
- PostgreSQL Row-Level Security
- WorkOS documentation

My final verdict

Lore

The Organizational Memory Platform

The product vision:

A platform that understands how an organization works, captures why important decisions are made, connects people with systems and knowledge, and helps employees access that experience when they need it.

The core product loop:

Observe → Understand → Capture → Verify → Connect → Assist

My six strongest recommendations:

1. Own the canonical knowledge model. Use Backstage and other standards as adapters, not as your entire architecture.
2. Build the GitHub integration first. It gives Lore structured, contextual engineering events.
3. Keep voice at the center. It is the mechanism that captures the missing reasoning.
4. Make permissions and provenance foundational. These determine whether organizations can trust Lore.
5. Start with PostgreSQL and pgvector. Avoid premature infrastructure complexity.
6. Build the Knowledge Continuity Engine. This is the long-term product vision that connects passive ingestion, proactive capture and contextual assistance.

The biggest strategic shift is this:

Lore should not just remember what an organization knows. It should help the organization recognize what it needs to remember next.

That is the direction I would build toward.